**FACULDADE DE TECNOLOGIA DE VOTORANTIM – FATEC**

CURSO SUPERIOR DE TECNOLOGIA EM DESENVOLVIMENTO DE SOFTWARE MULTIPLATAFORMA

DISCIPLINA DE MINERAÇÃO DE DADOS

**DANIEL RODRIGUES**

**EDUARDO KAMO**

**IAGO YURI**

**LUCAS VINICIOS CONSANI**

# ESTUDO DE CASO: DATA MINING EM TELECOMUNICAÇÕES

Arquitetura Multicloud | Spark | Flink | MinIO | Machine Learning

Plataformas Linux | Windows | Virtualização | Contentores

**VOTORANTIM – SP**

**2026**

# ESTUDO DE CASO: DATA MINING EM TELECOMUNICAÇÕES

Documento Oficial: Estudo_Caso_Real_Data_Mining_Telecom_14Out2026-v2

Trabalho prático avaliativo apresentado à disciplina de Mineração de Dados do Curso Superior de Tecnologia em Desenvolvimento de Software Multiplataforma da FATEC Votorantim, contendo a análise arquitetural multicloud e a resolução integral das questões operacionais.

**Docente:** Prof. Me. Benir Falcão Pistille.

**14 de Outubro de 2026**

# SUMÁRIO


**1. INTRODUÇÃO** ..... 4

**2. ARQUITETURA DE DATA MINING** ..... 4

2.1 Camada de Ingestão e Streaming ..... 5

2.2 Camada de Armazenamento Multicloud ..... 5

2.3 Camada de Processamento e Análise ..... 6

2.4 Camada de Governança e MLOps ..... 6

**3. ARQUITETURA MULTICLOUD** ..... 6

3.1 Kubernetes como Plataforma Unificada ..... 6

3.2 Virtualização e Contentores ..... 7

3.3 Rede e Conectividade ..... 7

**4. PIPELINE DE MACHINE LEARNING** ..... 7

4.1 Fluxograma Lógico do Pipeline ..... 7

4.2 Churn Prediction (Predição de Cancelamento) ..... 8

4.3 Deteção de Anomalias de Rede ..... 8

4.4 Propensão a Upsell ..... 8

4.5 Processamento Batch vs. Real-Time (Lambda Architecture) ..... 8

**5. CASO REAL: OPERADORAS GLOBAIS** ..... 9

5.1 Vodafone: Transformação AIOps Multicloud ..... 9

5.2 Operadora Asiática: Churn Prediction em Escala ..... 9

5.3 Ericsson Cloud Native Infrastructure ..... 9

5.4 Comparativo de Implementações ..... 10

**6. FERRAMENTAS OPEN SOURCE** ..... 10

**7. AUTOMAÇÃO E ADMINISTRAÇÃO** ..... 11

7.1 Infrastructure as Code (IaC) ..... 11

7.2 GitOps e CI/CD ..... 11

7.3 Automação de Rede SDN/NFV ..... 11

7.4 Monitorização Unificada ..... 11

**8. QUESTÕES DE PERFORMANCE, OTIMIZAÇÃO E CONTROLO (1 A 18)** ..... 11

# 1. INTRODUÇÃO

O setor de telecomunicações enfrenta desafios operacionais sem precedentes na era digital. Com biliões de transações de dados geradas diariamente por redes móveis, fixas e sensores IoT, as operadoras necessitam de arquiteturas robustas de processamento, análise e aprendizagem automática para extrair inteligência aplicável desses dados em tempo útil. Este estudo de caso apresenta uma arquitetura completa de Mineração de Dados para o setor de telecomunicações estruturada sobre um ecossistema multicloud e baseada exclusivamente em software livre (*open source*).

A arquitetura compreende desde a ingestão massiva de dados em escala petabyte — abrangendo Call Detail Records (CDRs), alarmes OSS, registos transacionais de CRM e transcrições de atendimento com Processamento de Linguagem Natural (NLP) — até à implementação de modelos de aprendizagem automática em contentores orquestrados por Kubernetes. Consideram-se sistemas operativos Linux (RHEL, Ubuntu), Windows Server, tecnologias de virtualização (KVM, VMware, Hyper-V) e orquestração de contentores (Docker, Podman, Kubernetes, OpenShift), operando de forma integrada entre AWS, Azure, GCP e datacenters próprios (*on-premises*).

# 2. ARQUITETURA DE DATA MINING

A arquitetura proposta é estruturada em camadas lógicas para garantir

separação de responsabilidades, alta disponibilidade, escalabilidade horizontal e ausência de dependência de fornecedores tecnológicos proprietários.

### 2.1 Camada de Ingestão e Streaming

Esta camada é a porta de entrada da plataforma, encarregada de recolher dados heterogéneos com volumes massivos e latências reduzidas. Em operadoras médias e de grande porte, os registos de chamadas e conexões de pacotes (CDRs) totalizam frequentemente entre 2 e 5 biliões de eventos por dia, com picos instantâneos que alcançam centenas de milhares de registos por segundo.

O Apache Kafka atua como a espinha dorsal de transmissão contínua (*streaming*), garantindo a replicação dos tópicos entre regiões geográficas e nuvens com particionamento robusto. O Kafka Connect estabelece a ponte com sistemas legados, bases NoSQL e lagos de dados. Em conjunto, o Apache NiFi gere fluxos complexos de ficheiros batch e logs heterogéneos, providenciando encaminhamento visual, encriptação e rastreamento de proveniência de dados em tempo real.

### 2.2 Camada de Armazenamento Multicloud

Adota-se o paradigma de Data Lakehouse, unindo a flexibilidade e o custo reduzido do armazenamento de objetos à integridade transacional dos Data Warehouses tradicionais. O MinIO e o HDFS atuam como armazenamento de objetos escalável com conformidade estrita à API S3 da AWS, instaláveis tanto em hardware local como em contentores de nuvem privada.

Formatos abertos de tabelas analíticas, nomeadamente Apache Iceberg e Delta Lake, garantem transações atómicas (propriedades ACID), isolamento de escrita concorrente, evolução transparente de esquemas e consulta a versões históricas (*time-travel*). Motores SQL distribuídos, como PrestoDB e Trino, executam consultas analíticas federadas de baixa latência diretamente sobre ficheiros colunares Parquet comprimidos com Snappy. O Apache Cassandra e o Redis oferecem suporte a

camadas de baixa latência e estado operacional para pontuação preditiva online.

### 2.3 Camada de Processamento e Análise

O Apache Spark é o motor central para computação paralela distribuída, processando tarefas em lote volumosas e operações em micro-lotes através do Spark Structured Streaming sobre a mesma base de código. Para necessidades de processamento de fluxos contínuos de latência sub-segundo, integra-se o Apache Flink.

A stack analítica baseia-se em Python (Scikit-Learn, XGBoost, LightGBM) para modelação tabular clássica e deteção de padrões de cancelamento de planos. Workloads de aprendizagem profunda (Deep Learning) utilizam TensorFlow e PyTorch distribuídos em nós acelerados por GPU em clusters Kubernetes. O AutoML livre (H2O-3 e AutoGluon) acelera a calibração de modelos estatísticos e a escolha ótima de hiperparâmetros.

### 2.4 Camada de Governança e MLOps

O ciclo de vida dos modelos é padronizado pelo MLflow, responsável pelo versionamento de experiências, hiperparâmetros e registo formal de artefactos. O Data Version Control (DVC) assegura a rastreabilidade dos conjuntos de dados pesados e pipelines. A validação de conformidade e integridade dos dados na entrada dos pipelines é efetuada de modo determinístico pelo Great Expectations. Em produção, a ferramenta Evidently AI monitoriza o desvio estatístico dos dados (*data drift*) e das predições (*concept drift*).

## 3. ARQUITETURA MULTICLOUD

### 3.1 Kubernetes como Plataforma Unificada

O Kubernetes (K8s) estabelece a camada de abstração operacional unificada,

operando clusters em serviços geridos de nuvem (AWS EKS, Azure AKS, Google GKE) e em instâncias bare-metal locais através de OpenShift ou Rancher. O Istio e o Envoy constituem a malha de serviços (*service mesh*), assegurando comunicação encriptada mTLS, encaminhamento inteligente de tráfego, gestão de falhas e observabilidade transparente.

### 3.2 Virtualização e Contentores

A infraestrutura organiza-se em três níveis essenciais de execução: (1) Máquinas virtuais tradicionais (KVM, VMware vSphere, Hyper-V) para alojar funções virtuais de rede legadas (VNFs); (2) Nós Windows Server no Kubernetes dedicados a manter aplicações legadas desenvolvidas no ecossistema .NET corporativo; e (3) Nós Linux (RHEL, Ubuntu) para todos os microsserviços modernos de ingestão, motores de processamento Spark e APIs analíticas.

### 3.3 Rede e Conectividade

Plugins de interface de rede (CNI) como Calico e Cilium gerenciam a segurança e a comunicação entre pods com elevado débito. A extensão Multus CNI possibilita a ligação de múltiplas interfaces de rede por contentor, requisito técnico fundamental para funções NFV. O MetalLB e o emparelhamento BGP interligam a infraestrutura dos clusters diretamente ao backbone IP/MPLS de telecomunicações.

## 4. PIPELINE DE MACHINE LEARNING

### 4.1 Fluxograma Lógico do Pipeline

O pipeline analítico é composto por 10 etapas sequenciais integradas em ciclo contínuo de feedback: (1) Recolha e consolidação de fontes; (2) Ingestão e streaming via Kafka/NiFi; (3) Armazenamento distribuído no Lakehouse; (4) Pré-processamento e limpeza de anomalias; (5) Validação automática com Great Expectations; (6) Engenharia de atributos temporais e grafos; (7) Divisão temporal sem contaminação

analítica; (8) Treino e modelação algorítmica; (9) Avaliação e explicabilidade via SHAP e curvas AUC; (10) Implementação em produção (*serving*) com KServe e monitorização contínua.

### 4.2 Churn Prediction (Predição de Cancelamento)

O modelo preditivo de evasão de assinantes emprega o algoritmo XGBoost treinado sobre mais de 150 atributos agregados em janelas temporais de 7, 30 e 90 dias. As variáveis preditivas englobam: taxa de chamadas caídas (*call drop*), degradação de sinal rádio (RSRP/SINR), volume de chamadas ao suporte, atraso de faturas e centralidade no grafo de chamadas telefónicas (PageRank e propagação de rótulos de evasão de contactos próximos). A divisão temporal estrita assegura a validação dos modelos sem fuga de informação. O modelo atinge AUC de 0.92 e precisão de 0.96 no topo dos 5% de clientes de maior risco.

### 4.3 Deteção de Anomalias de Rede

Modelos de Isolation Forest e autoencoders monitorizam em tempo real séries cronológicas de telemetria das células de transmissão móvel. Os alertas antecipados reduzem o tempo médio de reparação (MTTR) em 30%, antecipam 85% das falhas antes de reclamações formais dos assinantes e reduzem a sobrecarga do suporte em 40%.

### 4.4 Propensão a Upsell

Modelos baseados em LightGBM avaliam o comportamento de consumo e a proximidade do esgotamento da franquia de dados móveis, identificando os assinantes mais recetivos a migrações de plano e serviços de valor agregado. As campanhas ativadas no topo de propensão aumentam o CTR em 25%, as conversões em 18% e o ARPU médio em 12%.

### 4.5 Processamento Batch vs. Real-Time (Lambda Architecture)

A arquitetura adota a metodologia Lambda: a camada em lote (*batch layer*) processa históricos massivos em HDFS/MinIO com Apache Spark para treinar modelos e consolidar indicadores; a camada em tempo real (*speed layer*) processa fluxos contínuos no Flink e Kafka Streams para scoring imediato e bloqueio de fraudes; a camada de serviço (*serving layer*) unifica as perspetivas e expõe painéis operacionais no Apache Superset e Grafana.

## 5. CASO REAL: OPERADORAS GLOBAIS

A fundamentação do estudo ampara-se em implementações em operadoras globais de telecomunicações:

**Vodafone (Transformação AIOps Multicloud):** Em parceria multinuvem, migrou sistemas OSS legados para uma plataforma nativa na nuvem que consolida mais de 2 PB de dados operacionais e de rede, atingindo uma redução planeada de 50% em ferramentas proprietárias de monitorização e otimização automatizada de investimento em infraestrutura.

**Operadora Asiática (Churn Prediction em Escala):** Implementou um ecossistema com Apache Spark e Hadoop tratando 2.3 TB de dados novos por dia (97% gerados por redes OSS), atingindo 96% de precisão na previsão de cancelamento dos 50.000 clientes prioritários.

**Ericsson (Cloud Native Infrastructure):** Distribuição de contentores Kubernetes para funções 5G Core, reduzindo os tempos de atualização e implementação contínua (CI/CD) de semanas para segundos e gerando até 90% de economia nos processos de teste e integração.

| Operadora | Plataforma Central | Resultado de Negócio / ML | Tecnologias Principais |
|---|---|---|---|
| **Vodafone** | Google Cloud / Multicloud | 50% de redução de ferramentas OSS | AI/GenAI, BigQuery, Kafka |
| **Operadora Ásia** | Hadoop / Apache Spark | Precisão de 96% no topo de risco de churn | Spark SQL, HDFS, Random Forest, GBDT |
| **Ericsson** | Cloud Native Kubernetes | Economia de 90% em pipelines CI/CD | Kubernetes, Istio, Envoy, Kafka |
| **STL (Índia)** | CNCF / OpenShift | BSS cloud-native e modernização de faturação | Docker, Helm, Elastic Stack |

## 6. FERRAMENTAS OPEN SOURCE

O ecossistema é composto integralmente por ferramentas abertas consolidadas:

• **Ingestão e Streaming:** Apache Kafka, Kafka Connect, Apache NiFi, Logstash.

• **Armazenamento e Lago de Dados:** MinIO, Apache Iceberg, Delta Lake, HDFS, Apache Cassandra, Redis.

• **Processamento e Consulta:** Apache Spark, Apache Flink, Trino (PrestoDB), Apache Hive.

• **Inteligência e MLOps:** Scikit-Learn, XGBoost, LightGBM, TensorFlow, PyTorch, MLflow, Great Expectations, Evidently AI, KServe.

• **Orquestração e Infraestrutura:** Kubernetes, Docker, Podman, OpenStack, KVM, Terraform, Ansible, ArgoCD.

• **Observabilidade:** Prometheus, Grafana, Jaeger, Pilha ELK (Elasticsearch, Kibana),

Zabbix.

## 7. AUTOMAÇÃO E ADMINISTRAÇÃO

**7.1 Infrastructure as Code (IaC):** O Terraform aprovisiona a infraestrutura de computação, redes e storage na AWS, Azure, GCP e OpenStack com estado remoto e seguro. O Ansible realiza a configuração automatizada dos nós do sistema operacional e hardening de segurança.

**7.2 GitOps e CI/CD:** O ArgoCD aplica o estado declarado nos repositórios Git corporativos aos agrupamentos Kubernetes. Pipelines de CI/CD automatizadas compilam e validam contentores e enviam as imagens assinadas para o Harbor Registry com verificação de vulnerabilidades.

**7.3 Automação de Rede SDN/NFV:** Plataformas como ONAP (Open Network Automation Platform) e OpenDaylight realizam a orquestração e reconfiguração dinâmica de rotas e largura de banda na rede móvel.

**7.4 Monitorização Unificada:** O Zabbix supervisiona equipamentos legados e servidores físicos; o Prometheus e o Grafana monitorizam o ecossistema cloud-native e latências de microsserviços; a pilha ELK consolida o armazenamento e consulta de logs.

## 8. QUESTÕES DE PERFORMANCE, OTIMIZAÇÃO E CONTROLO

### Questão 1: Performance de Ingestão

*Uma operadora gera 5 bilhões de CDRs diários (média 58.000 registros/segundo com picos de 500.000/s). Proponha uma arquitetura de ingestão usando Apache Kafka que garanta throughput sustentado, tolerância a falhas e ordem de entrega para eventos do mesmo assinante. Considere particionamento por subscriber_id, replicação cross-AZ e balanceamento de carga entre*

*produtores.*

**Resposta:** Implementa-se um agrupamento distribuído de Apache Kafka estendido por 3 Zonas de Disponibilidade (AZs), utilizando fator de replicação replication.factor=3 e confirmação mínima de sincronização min.insync.replicas=2. Os produtores de dados são configurados com confirmação estrita (acks=all) e idempotência ativada, impedindo perdas ou gravações duplicadas. O particionamento dos tópicos de CDR utiliza obrigatoriamente a chave subscriber_id, garantindo que todas as chamadas e eventos de um mesmo utilizador fiquem estritamente na mesma partição e ordem cronológica. O dimensionamento contempla partições suficientes distribuídas pelos brokers para suportar com folga o pico de 500.000 eventos/s, tirando partido de compressão Zstandard em lote e balanceamento de produtores com verificação contínua de latência de consumo.

### Questão 2: Otimização de Storage

*O data lake da operadora cresce 50 TB por mês, com 80% dos acessos concentrados nos últimos 90 dias. Proponha uma estratégia de tiering multicloud (hot/warm/cold) usando S3 (standard/IA/Glacier) e MinIO on-premises, que otimize custos mantendo SLA de acesso de < 5 minutos para dados warm.*

**Resposta:** Classifica-se o ciclo de armazenamento por políticas automáticas de expiração e movimentação (lifecycle):

1. *Camada Hot (1 a 30 dias):* Armazenada em instâncias MinIO locais de alta velocidade ou S3 Standard para consultas e pipelines diárias imediatas.

2. *Camada Warm (31 a 90 dias):* Transferida para S3 Standard-IA ou MinIO com discos de maior densidade, preservando acesso em milissegundos e cumprindo folgadamente o SLA exigido de menos de 5 minutos.

3. *Camada Cold (acima de 90 dias):* Arquivada em S3 Glacier Flexible Recovery ou Glacier Deep Archive. Os dados são estruturados no formato colunar Parquet comprimido com Snappy e particionados por data, otimizando os custos de armazenamento e evitando despesas excessivas de recuperação.

### Questão 3: Latência de Streaming

*O sistema de detecção de anomalias de rede requer latência ponta-a-ponta < 2 segundos desde a geração do KPI até o alerta. Descreva como otimizar cada etapa: coleta SNMP/Kafka, processamento Spark Streaming/Flink, scoring do modelo (KServe) e notificação (WebSocket/Push).*

**Resposta:** Estabelece-se um orçamento temporal por etapa para assegurar latência global inferior a 2 segundos:

• *Coleta e Ingestão (SNMP para Kafka):* Alocação de até 300 ms, usando agentes leves com micro-lotes curtos e timestamps sincronizados.

• *Processamento Contínuo (Apache Flink):* Alocação de até 700 ms com janelas por tempo de evento, checkpoints assíncronos e armazenamento de estado em RocksDB local.

• *Pontuação Preditiva (KServe):* Alocação de até 300 ms, com réplicas aquecidas em contentores, modelos otimizados com ONNX Runtime e variáveis mantidas em cache Redis local.

• *Notificação e Envio (WebSocket / Push):* Alocação de até 300 ms para entrega direta nas consolas do NOC, mantendo uma margem de segurança de 400 ms para variações de rede.

### Questão 4: Controle de Transações

*O pipeline de churn prediction executa diariamente como job Spark batch. Como garantir exactly-once semantics para a atualização das tabelas de features e exportação das listas de intervenção para o CRM, evitando duplicação ou perda de clientes no top-5% de risco?*

**Resposta:** A execução do job diário é identificada de forma determinística por um identificador exclusivo (run_id + data de referência). A atualização das tabelas no Lakehouse adota Apache Iceberg ou Delta Lake utilizando a instrução MERGE INTO orientada pela chave subscriber_id, garantindo atomicidade e evitando inserções duplicadas caso ocorra reprocessamento do lote. Para o envio ao CRM, implementa-se o padrão de desenho *Transactional Outbox* com exportador idempotente: a lista versionada do topo de 5% é gerada com hashes únicos e enviada com chave de idempotência para o

endpoint do CRM, assegurando que envios repetidos decorrentes de falhas de rede sejam reconhecidos e ignorados pelo sistema recetor.

### Questão 5: Escalabilidade Kubernetes

*Durante eventos de alta audiência (ano-novo, black friday), o tráfego de dados cresce 10x. Descreva a configuração de HPA (Horizontal Pod Autoscaler) e VPA (Vertical Pod Autoscaler) para os serviços de scoring, incluindo métricas customizadas (Kafka lag, fila de requests) e cluster autoscaler para nodes GPU.*

**Resposta:** O Horizontal Pod Autoscaler (HPA) é calibrado para responder a métricas operacionais obtidas pelo Prometheus Adapter, monitorizando prioritariamente a profundidade das filas e o atraso de consumo (*Kafka consumer lag*) e pedidos por segundo, operando com scale-up rápido e scale-down amortecido. O Vertical Pod Autoscaler (VPA) permanece ativo em modo de recomendação durante a operação regular para evitar disputas de ajuste com o HPA. O Cluster Autoscaler é configurado com grupos dedicados de nós com GPU, aprovisionando máquinas adicionais assim que novos pods de inferência entram em estado pendente. Realizam-se ensaios de stress prévios e aquecimento preventivo de contentores antes do início de eventos de grande audiência.

### Questão 6: Otimização de Features

*A engenharia de features atual processa 2.3 TB/dia em Spark, consumindo 80% do tempo do pipeline. Identifique 3 otimizações técnicas (ex: broadcast join para tabelas pequenas, particionamento por data, reparticionamento após filter, cache de features frequentes no Redis) e estime o ganho de performance.*

**Resposta:** Aplicam-se três melhorias fundamentais:

1. *Broadcast Join:* Junções com tabelas dimensionais pequenas (planos, torres) utilizam envio em difusão (broadcast), suprimindo transferências pesadas de dados pela rede (shuffle), com ganho estimado de 20% a 50% nessas etapas.

2. *Particionamento por Data e Predicate Pushdown:* Armazenamento em Parquet particionado, permitindo leitura seletiva apenas das partições temporais necessárias e

reduzindo operações de disco (I/O) em 20% a 40%.

3. *Filtragem Antecipada e Cache Seletivo:* Eliminação precoce de registos corrompidos antes das transformações e armazenamento em memória/Redis de variáveis recalculadas com frequência, com ganho de 10% a 30%. O benefício total esperado é uma redução de 30% a 50% no tempo de processamento do pipeline diário.

### Questão 7: Multicloud Failover

*A arquitetura opera em AWS (primário) e Azure (DR). Descreva a estratégia de failover automático para o serviço de scoring de churn caso a região AWS fique indisponível, incluindo DNS (Route53 health checks), replicação de modelos (MLflow + S3 cross-region) e consistência de dados (Cassandra multi-DC).*

**Resposta:** Implementa-se uma contingência morna (*warm standby*):

• *Encaminhamento Global de DNS:* O Route 53 (ou balanceador multicloud) monitoriza a integridade dos endpoints da AWS através de verificações contínuas de saúde; constatada a indisponibilidade, o tráfego é comutado automaticamente para o Azure.

• *Sincronização de Modelos:* O repositório central do MLflow sincroniza os artefactos e configurações dos modelos homologados através de replicação automática de objetos de armazenamento entre a AWS e o Azure, validando a integridade por checksum.

• *Base Distribuída:* O Apache Cassandra é configurado com suporte a múltiplos datacenters (multi-DC), replicando dados de forma assíncrona entre AWS e Azure com consistência local (LOCAL_QUORUM), prevenindo perda de histórico e viabilizando a continuidade da operação sem bloqueios.

### Questão 8: Controle de Qualidade

*Great Expectations identifica que 15% dos registros CDR chegam com campos nulos em tower_id nas ultimas 24h. Proponha um pipeline de qualidade que: (a) bloqueie o job de features até resolução; (b) aplique imputação baseada em geolocalização do subscriber; (c) alerte a equipe de NOC via PagerDuty.*

**Resposta:** A validação no Great Expectations estabelece a regra expect_column_values_to_not_be_null com limite máximo de tolerância de 1%.

(a) Atingido o índice de 15% de valores nulos, a execução é interrompida imediatamente, impedindo a publicação de tabelas inconsistentes nas camadas analíticas e isolando os registos inválidos numa fila de quarentena.

(b) Inicia-se um fluxo paralelo de imputação: cruza-se o histórico de geolocalização recente do utilizador com o mapa geográfico das antenas circunvizinhas para estimar a antena mais plausível, mantendo o campo nulo caso persista ambiguidade e assinalando o registo com uma marcação de valor imputado.

(c) O sistema despacha de imediato um alerta de alta prioridade via API para o PagerDuty da equipa de operações de rede (NOC), indicando o lote com falha, o volume afetado e o procedimento de contingência.

### Questão 9: Performance de Modelo

*O modelo XGBoost de churn leva 800ms para inferência em produção, excedendo o SLA de 200ms. Identifique técnicas de otimização: quantização (INT8), pruning, ONNX runtime, batch inference, e caching de features por subscriber. Ordene por impacto esperado e facilidade de implementação.*

**Resposta:** A ordenação prática por facilidade e impacto provável define-se da seguinte forma:

1. *Cache de Atributos em Redis (Impacto Elevado / Facilidade Alta):* Guarda as variáveis agregadas já calculadas dos clientes, evitando recálculos repetidos em cada chamada e garantindo resposta em poucos milissegundos.

2. *Conversão para ONNX Runtime (Impacto Elevado / Facilidade Média):* Converte as árvores do XGBoost para a representação aberta e otimizada de baixo nível do ONNX, aproveitando acelerações de CPU/GPU sem perda de fidelidade matemática.

3. *Micro-lotes (Batch Inference) (Impacto Médio / Facilidade Média):* Agrupa pedidos simultâneos em janelas curtas de espera, reduzindo o custo por inferência.

4. *Poda de Árvores (Pruning) e Limitação de Profundidade (Impacto Médio / Facilidade Alta):* Reduz o número de nós com contribuição marginal para o resultado através de novo ajuste rápido.

5. *Quantização INT8 (Impacto Variável / Facilidade Baixa):* Requer validação cuidadosa de suporte de hardware e verificação contínua contra degradação de precisão estatística.

### Questão 10: Otimização de Rede

*O service mesh Istio adiciona 15ms de overhead por requisição entre microservicos de scoring. Proponha otimizações: mTLS permissive mode para tráfego interno, sidecar resource limits, eBPF-based Cilium para substituir iptables, e keep-alive connections. Estime a redução de latência.*

**Resposta:** Implementam-se quatro melhorias de conectividade:

• *Ligações Permanentes (Keep-Alive / HTTP/2):* Reutilizam canais de comunicação estabelecidos, eliminando o tempo associado à abertura contínua de sessões TCP e apertos de mão TLS.

• *Dimensionamento de Recursos dos Sidecars Envoy:* Ajuste rigoroso de alocação de núcleos e memória para os proxies, prevenindo limitações forçadas de ciclos de CPU (throttling).

• *Cilium com eBPF:* Substitui o tratamento tradicional por regras iptables do sistema operacional, acelerando o encaminhamento de pacotes no kernel do Linux.

• *Segurança:* Mantém-se o mTLS no modo estrito em produção por exigências de conformidade, alcançando uma redução estimada entre 5 e 10 ms do tempo de latência total.

### Questão 11: Controle de Custos

*O ambiente multicloud gera fatura mensal de $450k. Identifique 3 otimizações de custo significativas: Spot instances para workloads batch, rightsizing de VMs/containers via Prometheus metrics, armazenamento lifecycle policies, e consolidação de ferramentas de monitoramento. Estime economia percentual.*

**Resposta:** Adoção de três medidas financeiras de alto impacto:

1. *Instâncias Spot/Preemptible:* Execução de rotinas pesadas de processamento em lote do Spark e pipelines de validação utilizando capacidade computacional ociosa dos

provedores de nuvem, gerando reduções de até 70% nessas tarefas específicas.

2. *Adequação de Recursos (Rightsizing):* Reajuste sistemático de limites de CPU e memória em contentores e instâncias virtuais com base nos registos reais de consumo apurados pelo Prometheus, eliminando capacidade sobredimensionada.

3. *Políticas de Ciclo de Vida de Armazenamento:* Transferência programada de ficheiros históricos do Lakehouse para classes económicas de armazenamento e eliminação periódica de dados temporários.

*Resultado Estimado:* Poupança média entre 15% e 30% da fatura mensal, o correspondente a uma redução entre 67.500 e 135.000 dólares todos os meses.

### Questão 12: Virtualização vs Containers

*A operadora mantém 40% dos workloads em VMs (legado .NET/Windows) e 60% em containers (Linux/cloud-native). Proponha uma estratégia de migração gradual para Kubernetes, incluindo: (a) Windows containers para apps .NET; (b) virtual kubelet para cargas bursting; (c) critérios de decisão VM vs container.*

**Resposta:** Estrutura-se um roteiro de migração progressivo:

(a) As aplicações legadas no ecossistema .NET que apresentem viabilidade técnica são encapsuladas em contentores Windows e orquestradas em nós dedicados de Windows Server no próprio cluster de Kubernetes.

(b) O Virtual Kubelet é integrado para desviar aumentos pontuais e sazonais de carga (*bursting*) para serviços de computação sem servidor de nuvem pública, evitando a manutenção de nós físicos permanentes ociosos.

(c) *Critérios de Decisão:* Contentores são a escolha mandatória para soluções desacopladas, escaláveis horizontalmente e orientadas a microsserviços; mantêm-se máquinas virtuais exclusivamente para sistemas de fornecedores fechados dependentes de versões rígidas do sistema operativo ou controladores de hardware proprietários.

### Questão 13: Otimização de Pipeline ML

*O treino do modelo de anomalias leva 6 horas em cluster Spark. Proponha otimizações: sampling*

*estratificado para dados históricos, feature selection via XGBoost importance antes do treino, distributed hyperparameter tuning (Hyperopt/SparkTrials), e treino incremental (warm start). Estime redução de tempo.*

**Resposta:** Implementa-se uma reformulação metodológica no pipeline:

• *Amostragem Estratificada:* Redução do conjunto histórico de dados em 50% a 70%, assegurando a representatividade estatística de anomalias raras sem prejuízo da taxa de acerto.

• *Seleção Prévia de Variáveis:* Eliminação de atributos irrelevantes com base no índice de ganho do XGBoost antes do treino profundo, reduzindo a complexidade de cálculo.

• *Ajuste Distribuído com Hyperopt:* Exploração paralela de combinações de hiperparâmetros com paragem antecipada de tentativas com convergência desfavorável.

• *Treino Incremental (Warm Start):* Ajuste dos novos parâmetros a partir da iteração estável do dia anterior. O tempo de execução diminui de 6 horas para cerca de 2 a 2,5 horas, representando uma redução de aproximadamente 60%.

### Questão 14: Segurança e Compliance

*A LGPD exige anonimização de dados pessoais e auditoria de acessos. Descreva controles técnicos: campo-level encryption no Kafka/at-rest, RBAC no K8s/HDFS, vault para credenciais (HashiCorp Vault), audit logging centralizado (ELK), e data masking para ambientes de desenvolvimento.*

**Resposta:** A plataforma estabelece conformidade com a legislação através dos seguintes controlos:

• *Criptografia Ponta a Ponta:* TLS ativado em todas as transmissões de rede, AES-256 para dados em repouso no MinIO/S3 e encriptação específica por campo (*field-level*) no Kafka para identificadores de assinantes.

• *Gestão de Identidade e Controlo de Acesso:* Implementação de RBAC estrito nas camadas do Kubernetes, catálogo de dados e repositórios Lakehouse com base no privilégio mínimo.

• *Guarda Centralizada de Chaves:* O HashiCorp Vault gere dinamicamente certificados, chaves simétricas e credenciais temporárias de ligação a bases de dados.

• *Auditoria e Rastreabilidade:* Todos os acessos e modificações são registados em logs imutáveis e indexados no Elasticsearch para auditoria.

• *Mascaramento de Dados:* Ambientes de testes e validação operam unicamente com dados pseudonimizados ou gerados artificialmente através de processos estocásticos.

### Questão 15: Observabilidade e SLOs

*Defina SLOs para o sistema de predição de churn: disponibilidade (> 99.9%), latência de scoring (< 200ms p99), freshness de dados (< 24h), e accuracy mensal (AUC > 0.90). Descreva como Prometheus + Grafana monitoram cada SLO, com alertas graduados (warning/critical) e runbooks automatizados.*

**Resposta:** Fixam-se os seguintes compromissos de serviço:

• *Disponibilidade do Serviço:* Superior a 99.9% medida mensalmente.

• *Latência de Resposta:* Percentil 99 (p99) inferior a 200 ms.

• *Atualidade das Variáveis (Freshness):* Intervalo máximo de 24 horas entre a extração e a disponibilização dos dados.

• *Qualidade do Modelo:* Métrica AUC superior a 0.90 em avaliações com amostras rotuladas.

O Prometheus extrai métricas contínuas dos serviços e o Grafana exibe os painéis com a evolução dos orçamentos de erro aceitáveis (*error budgets*). Alertas do tipo Warning são ativados ao consumir 50% do orçamento de tolerância; alertas Critical são despachados ao PagerDuty na iminência de quebra de SLA. O MLflow aciona procedimentos automatizados de rollback para o modelo estável anterior caso a métrica AUC decaia.

### Questão 16: Apache Spark

*Citar 3 benefícios do Apache Spark*

**Resposta:**

1. *Processamento Distribuído em Memória RAM:* Distribui grandes volumes de

dados pela memória de múltiplos nós, permitindo desempenhos substancialmente mais rápidos que paradigmas antigos dependentes de leitura em disco.

2. *Plataforma Computacional Integrada:* Suporta rotinas analíticas em lote (Spark SQL), fluxos de dados contínuos (*Structured Streaming*), algoritmos de aprendizagem automática (MLlib) e análise de grafos sobre a mesma arquitetura de nós.

3. *Escalabilidade e Tolerância a Falhas:* Capacidade de ajuste dinâmico do número de trabalhadores e recuperação automática de partições corrompidas com suporte nativo a formatos de armazenamento modernos.

### Questão 17: Apache Flink

*Citar 3 benefícios do Apache Flink*

**Resposta:**

1. *Processamento Contínuo Orientado a Eventos (True Streaming):* Processa eventos unitariamente em tempo real com latências nativas na escala dos milissegundos, fundamental para alarmística de rede crítica.

2. *Gestão Confiável de Estado (Stateful Processing):* Assegura semântica estrita de processamento exato de cada evento (*exactly-once*) por meio de mecanismos periódicos e assíncronos de salvaguarda de estado (*checkpoints* e *savepoints*).

3. *Tratamento Temporal Avançado:* Suporte robusto a marcas de água (*watermarks*) e janelas dinâmicas com base no tempo de ocorrência do evento, permitindo reconciliar pacotes fora de ordem ou com atrasos de propagação na rede móvel.

### Questão 18: Plataforma MinIO

*Resumir os principais benefícios do MinIO, dentro de ambientes de larga escala para camadas de Object Storage (BLOB) e suas integrações com framework como Apache*

**Resposta:** O MinIO oferece uma infraestrutura de armazenamento de objetos distribuída, leve e de alto débito, com compatibilidade estrita à API S3. Permite

implementação em servidores bare-metal, nós virtualizados ou clusters Kubernetes em instalações locais e configurações multinuvem, eliminando a dependência de fornecedores proprietários e reduzindo significativamente os custos de licenciamento em escala petabyte.

Em ecossistemas de grande porte, oferece funcionalidades vitais como codificação de apagamento (*erasure coding*) para proteção contra falhas simultâneas de discos, versionamento de objetos, encriptação avançada e replicação geográfica ativa. A conformidade total com o padrão S3 possibilita a integração nativa com os principais frameworks do ecossistema Apache (Spark, Iceberg, Hive, Flink) e motores SQL federados (Trino/PrestoDB), tornando viável ler e persistir tabelas analíticas colunares de alto desempenho de forma direta e sem a sobrecarga de sistemas de ficheiros legados.